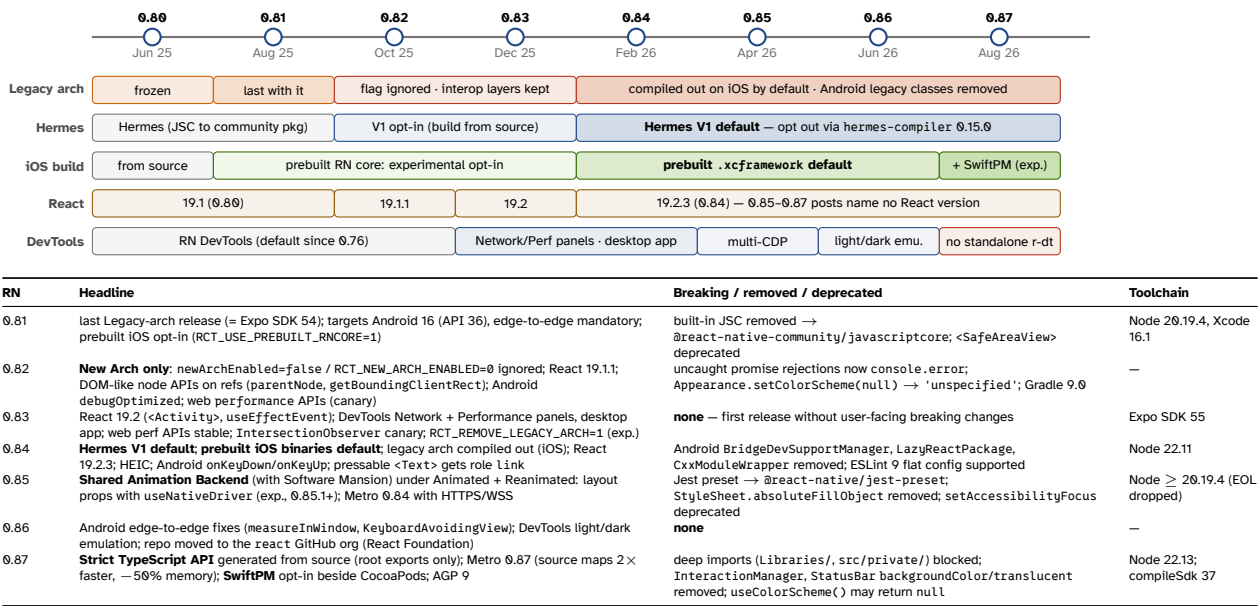


In one line: A minor about every **two months**; the current and the two previous minors are supported. **0.82** (Oct 2025) runs **only the New Architecture**; later releases remove the legacy code, make **Hermes V1** and **prebuilt iOS binaries** the default (0.84), share one **animation backend** (0.85), and generate a **strict TypeScript API** from source (0.87). 0.83 and 0.86 shipped **no user-facing breaking changes**.



Upgrade path

- **Still on the Legacy Architecture?** Official route: go to **0.81** (or Expo SDK 54), turn the New Architecture on, fix libraries there, *then* 0.82. Libraries not yet migrated run through the **interop layers**, kept “for the foreseeable future”.
- **Upgrade Helper** (react-native-community upgrade-helper) = the template diff between two versions: AppDelegate, Podfile, Gradle, package.json. Apply it by hand, one release at a time; read that release’s breaking-changes list.
- **Expo:** bump the SDK (it pins RN), then npx expo install --fix and npx expo-doctor. SDK 54 = RN 0.81; SDK 55 = RN 0.83.
- Bump native deps with RN (Reanimated, Screens, SVG...); they track the C++ APIs that each release removes.

In detail

- **DOM-like refs** (0.82): a host-component ref is a node with parentNode, children, getBoundingClientRect(), ownerDocument.getElementById(); measure/setNativeProps still work.
- **Web performance APIs** (stable 0.83): performance.mark/measure, PerformanceObserver, Event Timing, Long Tasks — measure real users, not just the lab.
- **Shared Animation Backend** (0.85): animation updates move into RN core, so Animated *and* Reanimated drive layout props (width...) natively.
- **SwiftPM** (0.87): only Xcode needed (no Ruby/CocoaPods); after a fresh clone and in CI run npx react-native spm once — the pod install analogue.

Example — the config lines these releases touch

```
# 0.84+: build RN core from source again (debug a C++ crash)
RCT_USE_PREBUILT_RNCORE=0 bundle exec pod install
# 0.82+: Android debug build with C++ optimised (~60 FPS)
npx react-native run-android --mode debugOptimized
// 0.85+: jest.config.js
preset: '@react-native/jest-preset',
// 0.87: tsconfig escape hatch (bridge through 0.88)
"customConditions": ["react-native",
  "react-native-legacy-deep-imports"]
# 0.87: SwiftPM instead of CocoaPods (experimental, once)
cd ios && npx react-native spm --deintegrate
```

Traps

- ! “0.82 deleted the legacy code” — it *ignores the flag*; class removal started after, and legacy libraries still load through the interop layer.
- ! Jumping 0.79 → 0.84 in one go: legacy arch, JSC, prebuilt binaries, Hermes V1 and Node 22 all at once — impossible to bisect.
- ! New crash reports after 0.82 may be old bugs: unhandled rejections stopped being silent.
- ! Hermes V1 on 0.82/0.83 needs RN built from source — not with prebuilt core.
- ! 0.87 breaks from 'react-native/Libraries/...' at type-check time.
- ! 0.87: useColorScheme() returns null, no longer 'unspecified' — a === 'unspecified' check silently stops matching.
- ! 0.85 tests fail to start until the Jest preset is renamed — not an app bug.

Remember

“82 new-arch only · 83 React 19.2 + DevTools · 84 Hermes V1 + prebuilt · 85 animation backend · 86 edge-to-edge · 87 strict TS + SwiftPM.”